

CardVault

Trading card collection manager. C237 Software Application Development, CA2.

Node.js + Express + EJS + MySQL.

Part 1 — Setup

Step 1: Tell Git who you are

This is important. It stamps your name on every commit you make.

In a terminal, run these two lines, with your own name and your school email:

```
git config --global user.name "Your Full Name"
git config --global user.email "your_student_id@myrp.edu.sg"
```

Check it:

```
git config --global user.name
git config --global user.email
```

It should print what you just typed.

Step 2: Download the project

Pick a folder where you keep your work — this example uses `C:\C237`. In the terminal:

```
cd C:\C237
git clone https://github.com/c237team4/CardVault.git
cd CardVault
```

Git may ask you to log in to GitHub. Follow the browser popup and using our team google account (look at our MS Team) to sign in.

Step 3: Install the project's libraries

```
# Run this inside your CardVault folder (C:\C237\CardVault)
npm install
```

This reads `package.json` and downloads Express, EJS, MySQL and the rest into a `node_modules` folder. It takes a minute and prints a lot of text. That's normal.

⚠️ **Do NOT run** `npm init -y`. That command is for starting a brand new project — it will **overwrite** `package.json` and wipe our dependency list, breaking the app for all six of us. We only ever run `npm install`.

You also don't need `npm install express`, `npm install ejs`, and so on one by one. That's for building a project from scratch. `npm install` on its own reads `package.json` and installs everything at the pinned versions.

`node_modules` **is not in Git on purpose**. It's thousands of files that anyone can regenerate with `npm install`, and the CA2 spec says the submission zip must exclude it. Never add it to Git.

Step 4: Run it

```
# Run this inside your CardVault folder (C:\C237\CardVault)
node app.js
```

You should see:

```
Server running on port 3000
Connected to MySQL database
```

Open <http://localhost:3000> in your browser. You should see the CardVault navbar.

Now stop the server, press `Ctrl + C` in the terminal.

If you got both lines and the page loads — setup is done.

The database — read this before you touch it

****MySQL ****database has already setup on Azure, and the login details are in `app.js`. If you saw `Connected to MySQL database`, you are already on it.

We all share ONE database. There is no copy on your laptop. When you add a card, everyone sees it. When you break something, everyone gets it.

The tables

Table	What's in it	Owner
<code>users</code>	accounts, roles	A
<code>cards</code>	user collection	B / C / D / E / F
<code>genres</code>	Trading Card Games, Sport Cards, Others	admin-curated
<code>conditions</code>	Mint → Poor, with <code>condition_rank</code> (1 = best)	admin-curated

You can refer to the screenshots posted in MS Team or login via MySQL Workbench

Test logins

Already in the database, ready to use:

Email	Password	What they have
ryan@demo.sg	demo123	Pokemon collector, 12 cards, ~S\$28k
mei@demo.sg	demo123	Sports collector, 12 cards, ~S\$5.6k
jun@demo.sg	demo123	Casual, 16 cards, ~S\$420
admin@cardvault.sg	admin123	Admin, no cards (use this to test empty states)

You can also register your own account at `/register`.

Part 2 — The daily routine

Four steps, every single time you sit down to work. Do them in order.

1. Pull first — ALWAYS

```
# Run this inside your CardVault folder (C:\C237\CardVault)
git pull
```

Do this before you write anything. It brings in your teammates' latest work. Skipping this step is the #1 cause of merge conflicts.

2. Do your work

Edit **only your own section**. See *Who owns what* below.

3. Test before you commit

```
# Run this inside your CardVault folder (C:\C237\CardVault)
node app.js
```

Check your page in the browser. **Never commit code that doesn't run** — it breaks the app for all five other people, and they'll lose time figuring out it wasn't their fault.

4. Commit and push

```
git add .
git commit -m "Change this comment to your own comment e.g Add search filter"
git pull
git push
```

Note the `git pull` *before* `git push`. If a teammate pushed while you were working, this pulls their changes in first. Without it, your push gets rejected.

Write commit messages that say what you did. "Add login validation" is useful.

Who owns what

Open `app.js` and scroll to the `ROUTES` section. It's split into six labelled blocks — **one per person**, already assigned:

```
// STUDENT B | Owner: Ryan
// Adding New Information -- add a card to your collection
// Routes: GET /add-card, POST /add-card
```

Find your name, and only add code inside your own block. Your name sitting above the code you wrote, plus your commits touching those lines, is what shows the lecturer which part was yours.

Your view files have the same header, with notes on what the route passes you and what the form needs. Read them before you start — they will save you an hour.

Block	Owner	Responsibility	Views you own
A	Boon Meng	Registration, Login, Access Control	<code>login.ejs</code> , <code>register.ejs</code>
B	Ryan	Adding a card	<code>add-card.ejs</code>
C	Sammi	Dashboard + viewing cards	<code>dashboard.ejs</code> , <code>view-collection.ejs</code>
D	Ezann	Editing a card	<code>edit-card.ejs</code>
E	Rainie	Deleting cards + admin (users, genres, conditions)	<code>admin-dashboard.ejs</code>
F	Zhan Fung	Search / filter / sort your own collection	<code>view-collection.ejs</code> (shared with C)

Why this matters: Git merges cleanly when people change *different lines*. It gets confused when two people change the *same* lines. Staying inside your own block means you can all work at the same time without fighting.

Two overlaps to sort out at the first meeting

C and F share `/view-collection`. C renders the collection; F makes it searchable. Same route, same view — so agree the split before either starts. Suggested: C owns the route and the view file, F owns the filter logic that C's query calls.

E owns the admin. That means the reference lists (`genres` , `conditions`) and user management. It is not optional decoration — the admin curating those lists is *why* F's filtering works, and it is how we demonstrate having two roles.

Shared files — coordinate before touching

These belong to everyone, so inform in the group chat before editing:

- `views/partials/header.ejs` — the navbar
- `views/partials/footer.ejs`
- The `SHARED` block in `app.js`
- `package.json`
- `database/cardvault.sql` — see the database warning above

When something breaks

Cannot find module 'express' (or any module)

You skipped `npm install` , or a teammate added a new library.

```
# Run this inside your CardVault folder (C:\C237\CardVault)
npm install
```

Error: listen EADDRINUSE: address already in use :::3000

The server is already running somewhere — usually an old terminal you forgot about.

Press `Ctrl + C` in any other terminal tabs. If that fails, close all terminals and open a new one.

Error connecting to MySQL

Usually the campus network or Wi-Fi. Check you're online. If your internet is fine and it still fails, ask the group — it may be the shared class database being down, which affects everyone.

Updates were rejected because the remote contains work that you do not have

A teammate pushed before you. Fix:

```
# Run these inside your CardVault folder (C:\C237\CardVault)
git pull
git push
```

CONFLICT (content): Merge conflict in app.js

You and a teammate changed the same lines. Git marks the clash like this:

```
<<<<<< HEAD
your version
=====
their version
>>>>>> main
```

Open the file, decide what the correct final code is, delete the `<<<<<<`, `=====` and `>>>>>>` marker lines, then:

```
git add .
git commit -m "Resolve merge conflict"
git push
```

If you're not sure which version is right, ask the person who wrote the other half before deleting anything. Don't guess — you might delete their afternoon.

warning: LF will be replaced by CRLF

Harmless. Windows and Git disagree about invisible line-ending characters. Ignore it.

I've broken everything and want to start over

Your last commit is safe. This throws away **uncommitted** changes only:

```
# Run this inside your CardVault folder (C:\C237\CardVault)
git checkout .
```

Rules

1. **git pull** before you start. Every time.
2. **Test before you commit.** If `node app.js` fails, don't push it.
3. **Stay in your own block.** Ask before touching shared files.
4. **Never commit** `node_modules` .
5. **Push often** — small commits beat one giant commit the night before. They're also your evidence of contribution.

6. **Understand your own code.** The lecturer will ask you to explain your feature: what it does, the route, the SQL, the database, the response. "AI wrote it" loses marks.
-

Quick reference

```
git pull                # get teammates' latest work – do this FIRST
npm install             # install libraries (after cloning, or if a module is missing)
node app.js            # run the app -> http://localhost:3000
                        # Ctrl + C to stop

git status             # what have I changed?
git add .              # stage my changes
git commit -m "message" # save them locally
git pull               # merge in anything new
git push               # send to GitHub

git log --oneline      # recent history
```